MTRF: Multidomain Transformation Representation for Network Flows in Network Intrusion Detection

Xinlei Wang, Mingshu He, *Member, IEEE*, Xiaojuan Wang, *Member, IEEE*, and Shize Guo

***Abstract*—In IoT device applications, due to privacy protection requirements, it is often impossible to obtain large-scale labeled datasets for model training. A representation model that effectively extracts flow features with limited labeled samples is therefore critical. To meet this need, we propose a model combining temporal features (for trend changes/short-term fluctuations) and frequency-domain features (for periodicity/stability patterns) in network flows. In order to fully tap the potential of these two types of features, contrastive learning (CL) technology is used to model them respectively. For temporal-domain features, we employ a momentum encoder-based training strategy to learn robust representations adaptable to network-induced noise. For frequency domain features, we propose a new supervised CL loss function. It enhances inter-class separability while improving the representation ability (i.e., the model's capacity to extract discriminative features) of minority classes. The experimental results demonstrate the universality of this approach, which is not limited to specific data sets or attack types. The most representative results were obtained on the UNSW-NB15 dataset, with an accuracy of 0.9699 for the balanced training sets, surpassing the other best models' performance(0.6300).**

***Index Terms*—Contrastive learning, frequency domain transformation, intrusion detection, sample representation.**

I. Introduction

INTERNET of Things (IoT) adoption escalates cyberattack risks in sensitive-data industries, compromising intelligent analysis and decision-making. While many intrusion detection models perform well, they are often deployed in the cloud, leading to detection delays and privacy risks [26]. By deploying Intrusion Detection Systems (IDS) closer to IoT devices at the network edge, fog computing mitigates latency issues inherent in cloud-centric approaches [25]. However, due to the limited resources of the fog equipment, a lightweight model with high detection accuracy is required. In this study, we propose a novel learning representation method for fog nodes, using a small set of labeled samples to characterize encrypted network flows. This representation supports the creation of lightweight IDS models, including Extra Tree (ET), Random Forest (RF), Decision Tree (DT), and eXtreme Gradient Boosting (XGBoost), tailored for IoT devices.

In recent years, significant efforts have been made in IDS, with most research focusing on framing intrusion detection as a classification problem and developing end-to-end models. Traditional machine learning (ML) algorithms offer fast inference speeds but rely on manual feature selection, limiting adaptability to emerging threats [19]. Although end-to-end DL models are widely studied [1], [3], [11], [12], [14], [16], their joint learning approach may overfit and capture spurious correlations from noisy network flow data [5]. Emerging malware targeting compromised IoT devices demands models that learn broader data representations. However, data collection and labeling difficulties often yield imbalanced datasets with scarce labeled examples. This highlights two critical challenges: capturing subtle behavioral differences in minority classes and improving generalization from limited data.

Representation learning captures intrinsic features of unlabeled data, improving generalization to new data [20], [21], [22]. Ceschin et al. [23] employed denoising autoencoders for unsupervised pre-training. Optimizing the encoder's representational capacity by minimizing the loss function between input samples and predicted outputs. [9] employs supervised contrastive learning (CL) for a binary representation model, but directly feeding features into the model without distinction may lead to overfitting noise. [24] suggests separate representation learning for categorical and numerical features, using a hard triplet loss for classes with limited labeled data. Based on this study, we were inspired to use hard contrastive loss on the labeled dataset to enhance the representation learning of minority class samples. However, in our approach, we aim to separately learn different types of features to mitigate the interference of noisy features on other types.

Complex network topologies, such as distributed systems, inevitably lead to network-induced phenomena, causing packet loss, transmission delays, and other issues [1]. The delay or

Received 12 January 2025; revised 24 July 2025; accepted 24 December 2025. Date of publication 29 December 2025; date of current version 12 May 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62402053 and Grant 62227805 and in part by the Fundamental Research Funds for the Central Universities under Grant 2025KYQD17 (BUPT). *(Xinlei Wang and Mingshu He contributed equally to this work.) (Corresponding author: Mingshu He.)*

Xinlei Wang is with the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with the The First Research Institute of the Ministry of Public Security of P.R.C, Beijing 100741, China (e-mail: ann96125@126.com).

Mingshu He is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with the Key Laboratory of Trustworthy Distributed Computing and Service (BUPT), Ministry of Education, Beijing 100876, China (e-mail: hemingshu@bupt.edu.cn).

Xiaojuan Wang and Shize Guo are with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: wj2718@bupt.edu.cn; nsfgsz@126.com).

We have released our source code to facilitate future studies on https://github.com/Ann96125/MTRF.

Digital Object Identifier 10.1109/TDSC.2025.3649110

interruption of network data packet transmission is manifested in the time domain as a feature with transmission noise. Alternatively, the frequency domain features of network traffic are less sensitive to interference and can better reflect the inherent properties of the data, such as periodic components. These stable features are less influenced by external disturbances, making them more reliable for analysis [6]. Separately learning both features ensures that interference in the temporal domain does not affect the representation learning of the frequency domain.

Under the assumption in [5], data X is generated by the error variable ϵ and the error-free latent variable X^*. Here, as the error ϵ varies, the latent variable X^* remains unchanged. Thus, learning the representation of X^* involves finding stable associations with the optimal X^* across various types of errors. Along these lines of thought, we propose a model called the *Multi-domain Transformation Representation of Network Flows* (MTRF). MTRF can simulate noise in different network environments and learn invariant representations of useful features.

The dataset was augmented eight times through network-induced perturbations, increasing the training samples eightfold. This augmentation with the MTRF method enhanced the stability of network flow representations despite the small sample size. In the temporal domain, self-supervised CL is used to construct a feature space. This involves maximizing the feature similarity of positive samples and minimizing the similarity of negative samples (i.e., noise interference). The result is disentangled representations of temporal domain features and noise, thereby enhancing the robustness of network flows against environmental interference.

Furthermore, using relatively stable frequency-domain features increases the distance between negative samples. Inspired by [24], we incorporate hard margin penalty into the learning of frequency domain features. The core of hard margin penalty is to enforce a strict constraint that mutually negative samples maintain a clear separation in the feature space. In [24], hard margin is incorporated into the contrastive loss, requiring negative samples to be at least M (where M is the set threshold) farther away from the anchor samples than positive samples. The aim is to bring the original data into a more precise representation space using a hard margin penalty factor, while introducing relaxation factors and regularization terms to mitigate the risk of overfitting in the representation. The contributions of our work are as follows:

1) We deploy the representation model on fog nodes. To prevent interference, we extracted the temporal-domain and frequency-domain features of network traffic separately. The represented test data is suitable for customized lightweight intrusion detection models in IoT devices.
2) MTRF uses original packet headers as input, avoiding feature selection and plaintext payloads for privacy protection. We apply a hard margin penalty factor for frequency-domain contrastive loss to enhance representations of minority classes.
3) We devised five tasks varying in sample distribution difficulty, covering balanced and imbalanced datasets of different sizes. Results showed MTRF excelling in both multi-class and binary classification, achieving the best F1 score of 0.9802 on the imbalanced MQTT dataset.

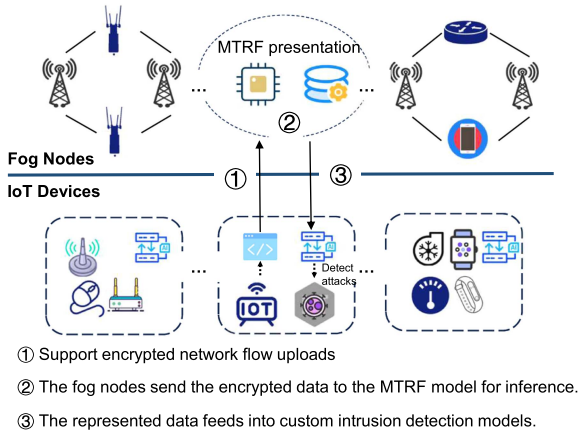


Fig. 1. Diagram illustrating fog nodes for data representation combined with rapid inference models on IoT devices.(The figure only shows the inference stage.

II. Proposed Framework

In this section, we present our fog based framework for detecting intrusions in IoT traffic. The process diagram for intrusion detection using fog and edge computing is illustrated in Fig. 1. Fog nodes deployed near devices extend IoT capabilities, supporting low-latency intrusion detection. These nodes can use encrypted network traffic and perform inference through the MTRF model, thereby avoiding the privacy leakage that occurs when IoT devices transmit data to fog nodes. The characterized flows obtained from the MTRF model are transmitted back to IoT devices for edge detection. Different IoT devices can flexibly choose detection models based on storage capacity, supporting lightweight ML detection models. With reduced sample dimensions after characterization by fog nodes, there are advantages such as suitability for lightweight device storage and detection, reduced transmission latency, resistance to reverse engineering, and enhanced privacy security.

The cloud server conducts training on a small number of network flows. Due to the small amount of training data required by MTRF, it avoids the increase in network bandwidth resources and transmission delay caused by uploading a large number of samples.

III. Related Work

A. Deep Learning-Based IDS

We summarize the IDS using DL models, categorizing them broadly into two types: end-to-end models and non-end-to-end representational models. In end-to-end models, CNNs have long been considered as good classification models. [3] treats network flows as images and, using Convolutional Neural Networks (CNNs) for classification, demonstrates that multi-layer bidirectional network protocols are optimal. [16] preprocessed pcap files with IP masking and other methods before directly using autoencoders and CNNs for encrypted traffic classification. [14] used CNNs, transfer learning, ensemble learning, and hyperparameter optimization for complex network flow data, adding hyperparameter tuning beyond prior research. [11] addressed

concept drift in evolving malware samples by updating feature categories. This indicates that different feature representations of the same dataset can have a significant impact. Therefore, it is highly necessary to propose a unified representation model to enhance task adaptability. [12] adopt feature engineering for network flows and use stacked tree models to train the meta-learner. Low-confidence samples are input into the biased classifier for detection. [1] enhance the robustness of network flows by simulating network-induced phenomena (e.g., packet loss) via recurrent neural networks. These end-to-end methods offer quicker detection but demands more feature engineering for input data.

Non-end-to-end models primarily refer to representing network flows before feeding them into classifiers. We roughly categorize the representation process into methods based on graphs, CL, and encoder-decoder approaches. [10] used topological data analysis to extract features before detection with ML models. [8] created graph representations of network flows with feature-rich nodes and edges, feeding them into Graph Neural Networks (GNNs) for training. Graph-based approaches essentially relies on feature engineering. [15] divide the input data into K groups and construct a topological structure for each group based on the clustering results. It is demonstrated that this method is highly effective for imbalanced datasets. [13] utilized a feature extraction encoder with enhanced representation capability and weighted supervised contrastive loss to address class-imbalanced learning. [31] developed a supervised variant of the twin variational autoencoder to enhance class discrimination by transforming latent representations into distinguishable Gaussians using labels. Compared to graph-based methods, CL and encoder-decoder approaches do not require predefined graph structures. Instead, they can automatically learn intrinsic data representations and capture important features directly from the raw input.

B. Related Works to CL

CL is a promising approach in unsupervised representation learning. The core idea of CL is to unearth semantic discriminative features by contrasting similarities and differences between samples, significantly reducing reliance on manually annotated labels. However, existing CL-based methods still face critical challenges. Current methods [27], [33], [29], [34] require large-scale datasets or strong distributional assumptions for training, limiting flexibility in real-world networks with scarce labels and dynamic distribution shifts. Besides, in some studies, the temporal sensitivity [35] and semantic complexity [28] of network traffic data are ignored. More critically, the existing multimodal models [30] often entangle the modal specificity (information unique to a certain modality) and modal sharing (information shared by multiple modalities) representations, which limits the classification performance of the models [36]. Table I summarizes the comparison between the existing CL methods and the method proposed in this paper.

[27] embeds labels as class prototypes into the same space as feature embeddings to reduce the number of pairwise comparisons. However, this method relies on fully labeled datasets, posing challenges for storage-constrained fog computing nodes. MLCL [33] employs three fusion strategies to achieve cross-modal semantic alignment between images and text. However, MLCL is validated only on large-scale datasets and does not address few-shot scenarios. [29] initializes weights based on class proportions in the dataset to improve minority-class performance. However, this requires consistent data distributions between training and testing sets. [34] combines supervised CL with self-supervision, but its 1,440-batch ImageNet pre-training requires multi-GPU parallelism – infeasible for resource-constrained scenarios. The USTGCL model [35] achieves data augmentation at topological and feature levels but overlooks temporal correlations. However, network traffic detection relies more on dynamic time-series features. [28] constructs a CL task by randomly masking network packet sequences. This method only considers packet loss and fails to cover more complex network environments (including packet retransmission, out-of-order delivery). [30] proposes a multimodal learning method based on federated learning. The reconstructed data of each modality is decoded from the shared representation encoded by the original input data. However, the shared representation may not fully capture each modality's specific information, leading to the loss of some modality-specific details or information during the decoding process.

TABLE I
THE KEY DIFFERENCES BETWEEN THE PROPOSED WORK AND EXISTING APPROACHES

	Existing approaches	Proposed work
[28]	Requires a large-scale fully-labeled dataset	Train with a small amount of labeled data
[34]		
[35]	Large batch size	
[30]	Train-test distribution consistency assumption	Test data contain unseen samples
[36]	Ignoring temporal features	Multi-kernel 1D convolutions capture multi-scale temporal patterns
[29]	Limited data augmentation	Four network augmentation modes
[31]	Inter-domain shared representation	Separate temporal and frequency representation learning

In summary, current CL-based research faces three key challenges: existing methods rely on large-scale labeled data or distributional assumptions, struggling to adapt to real-world networks with label scarcity and dynamic distribution shifts. Data augmentation for network flows must address temporal sensitivity and semantic complexity. Feature entanglement in cross-domain fusion introduces unpredictable noise. To overcome these, this paper proposes a non-end-to-end framework requiring minimal labeled data for enhanced transferability. Inspired by [1], we introduce four environmentally induced augmentations to learn representations separately in time and frequency domains.

C. The Principle of MoCo

This subsection will summarize the operation process of momentum contrast (MoCo) [32]. Given a sample x, generate two different views of it through data augmentation: the query view x_q and the key view x_k. The encoder f_q processes x_q to generate query feature $q = f_q(x_q)$; the momentum encoder

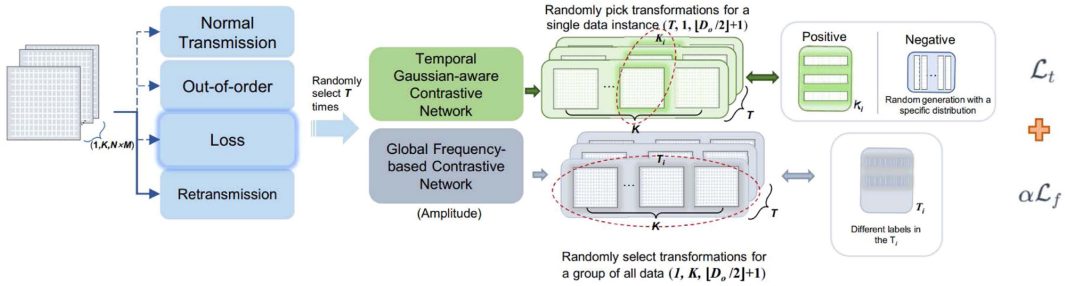


Fig. 2. Overall Framework.

f_k (whose parameters are the sliding average of f_q) processes x_k to generate key feature $k = f_k(x_k)$, and stores k in the queue. The queue stores a large number of key features from historical batches as negative samples (i.e., enhanced features of other samples), with the goal of bringing the distance between q and its positive sample key feature k closer through contrastive loss (such as InfoNCE), while pushing the distance between q and all negative sample key features in the queue farther apart. This is to update the parameters of f_q and improve its feature quality.

MoCo's dynamic queue stores historical negative key features (generated by the momentum encoder f_k). When the parameters of the query encoder f_q are updated rapidly, the output query feature q may undergo feature space drift. At this point, if the negative features in the queue are generated by an earlier version of f_k, the similarity discrepancies between q and the negatives may stem from feature space shifts between the new and old models, rather than the semantic differences of the samples themselves, rendering the contrastive loss ineffective. If f_k were updated simultaneously with f_q each time, the negative features in the queue would lack consistency—negatives from different moments would be generated by highly divergent versions of f_k—also leading to the aforementioned issue.

Thus, MoCo designs two techniques. The first one momentum update strategy. The parameters of f_k are updated via exponential moving average (EMA) toward those of f_q, preventing abrupt feature space shifts. The second one is the queue update strategy. When new key features (generated by f_k) from each batch are enqueued, the oldest batch of features is dequeued following a first-in-first-out (FIFO) policy. This ensures that the negative samples in the queue are always derived from recent versions of f_k, thus preserving the consistency of the feature space.

IV. Methodology

Given a labeled dataset $\mathcal{F} = \{(\mathbf{F}_k, y_k), k = 1, \ldots, K\}$ consisting of K network flows, each of which undergoes a series of random transformations, including normal transmission, packet reordering, loss, and retransmission. Subsequently, these transformed flows are individually inputted into the temporal-domain and frequency-domain networks to extract multi-scale features. Following that, these features are propagated to two distinct branches, including *Temporal Gaussian-aware Contrastive Network* (TGCN) branch and the *Global Frequency-based Contrastive Network* (GFCN) branch.Two branches train the same encoder, which is then used for inference representation. For the TGCN branch, MoCo is applied to compute the contrastive loss $\mathcal{L}_t$, using a momentum encoder for positive sample representation and a dynamic queue for negative sample learning. The encoder employs a multi-layer feature extractor to capture temporal patterns in $\mathcal{F}$. For the GFCN branch, the Fourier transform is used to extract the amplitude-frequency representation of $\mathcal{F}$, and a contrastive loss $\mathcal{L}_f$ with hard boundary parameters is applied to capture relationships between different classes. For the sake of clarity, Fig. 2 illustrates the overall architecture and training process of our proposed method. In summary, the primary objective of our approach is:

$$\mathcal{L} = \mathcal{L}_t + \alpha\mathcal{L}_f \tag{1}$$

where α represents the hyperparameter that regulates the equilibrium between the learnable temporal and frequency components. Below is a detailed description of the network flow preprocessing and the two branches.

As there are numerous symbol representations in this paper, Table II is provided for reference.

A. Packet Segmentation and Enhanced Transformations

The unidirectional flows are extracted by polling all packets in the packet capture (pcap) file with the same five-tuple. Each flow $\mathbf{F}_k$ consists of N packets, and the first M decimal digits from the packet headers are extracted. In this way, a single flow is represented as an $N \times M$ matrix. The matrix $\mathbf{F}_k$ represents the collection of information quantities for each packet $\mathbf{P}_i^{(k)} = [p_i^{(k,1)}, p_i^{(k,2)}, \ldots, p_i^{(k,j)}](j = 1, 2, \ldots, M)$, where $p_i^{(k,j)}$ represents the j-th decimal digit of the i-th packet.

$$\mathbf{F}_k = \begin{bmatrix} \mathbf{P}_1^{(k)} \\ \mathbf{P}_2^{(k)} \\ \vdots \\ \mathbf{P}_N^{(k)} \end{bmatrix} = \begin{bmatrix} p_1^{(k,1)} & p_1^{(k,2)} & \cdots & p_1^{(k,M)} \\ p_2^{(k,1)} & p_2^{(k,2)} & \cdots & p_2^{(k,M)} \\ \vdots & \vdots & \ddots & \vdots \\ p_N^{(k,1)} & p_N^{(k,2)} & \cdots & p_N^{(k,M)} \end{bmatrix} \tag{2}$$

Flatten each network flow $\mathbf{F}_k$ into a $1 \times (N \times M)$ vector $F_{k,t}(t = 1, 2, \ldots, N \times M)$. Construct matrix $D \in \mathbb{R}^{K \times (N \times M)}$ by stacking the flattened $1 \times (N \times M)$ vectors of all flows, with each row corresponding to a flow.

TABLE II
MATHEMATICAL SYMBOL TABLE

Symbol	Description	Symbol	Description
$\mathbf{F}_k$	The k-th network flow in the dataset	$\mathcal{B}$	Background matrix
$\mathbf{P}_i^{(k)}$	The i-th packet of the k-th network flow in the dataset	$\tilde{\mathcal{B}}$	DFT to matrix $\tilde{\mathcal{B}}$ along its flow dimension (dim=1)
D	Flattened matrix formed by stacking all flow vectors	$\mathcal{B}_L$	The $\tilde{\mathcal{B}}$ after affine transformation
D^T	D augmented T times	$\tilde{\mathcal{B}_L}$	DFT to matrix $\mathcal{B}_L$ along its feature dimension (dim=2)
θ_{base}	Base encoder for encoding anchor samples	$\mathcal{B}_A$	Extract the amplitude spectrum from $\tilde{\mathcal{B}_L}$ (from a certain augmentation)
θ_{mom}	Momentum encoder for encoding positive samples		
D_A^T	After the first encoding step by the θ_{base}, D^T is encoded	N_f	The similarity matrices for negative pairs
D_P^T	After the first encoding step by the θ_{mom}, D^T is encoded	$\mathcal{L}_f$	Frequency domain contrast loss
A_t, P_t, N_t	Calculate the anchor, positive, and negative samples of $\mathcal{L}_t$	$\mathcal{L}_t$	Temporal domain contrastive Loss

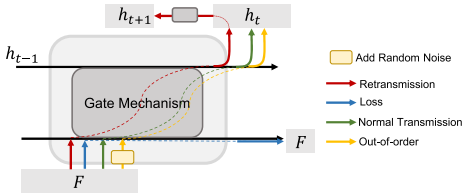


Fig. 3. The representation of the network-induced environment of data flows using a LSTM network. The gate mechanisms, which are consistent with the regular LSTM, are simply depicted here.

We utilize the method proposed in [1] to simulate common network-induced phenomena, including normal transmission, retransmission, loss, and out-of-order, in order to enhance the error resilience of network flows and improve the robustness of the model. Specifically, (1) Normal Transmission indicates that the network flow is directly processed through the traditional GRU network. (2) Retransmission represents performing the same operation again on the currently processed network flow, i.e., repeating the process through the GRU network. (3) Loss indicates skipping all network flows, preventing them from passing through the GRU network. (4) Out-of-order involves introducing random noise to each network flow before passing it through the GRU network. This is achieved by randomly adding noise to the bytes in the network flow (each processed byte should not exceed 255), approximating the combination of multiple random noise processes.

At the same time, we encapsulated the entire dataset and randomly selected one transformation method to establish a uniform network-induced environment for all data instances, enabling iterative environment transformations to handle small sample sets. The Fig. 3 illustrates the representations corresponding to the four states.

After T iterations of data augmentation, D_i represents the transformation of a dataset containing K flows triggered by a specific network environment. All matrices D_i are concatenated along the first dimension to obtain the final transformation result D^T.

$$D^T = \{D_1, D_2, \ldots, D_T\},\ D^T \in \mathbb{R}^{T\times K\times(N\times M)} \tag{3}$$

For the TGCN branch, the positive samples originate from the predefined tensor D^T, while the anchor samples are generated by replicating matrix D T times along the first dimension.

B. Temporal Gaussian-Aware Contrastive Network

In CL, the model's output undergoes loss computation through an instance discrimination proxy task, followed by backpropagation, making it a self-supervised learning approach. A MoCo [32] variant is applied to obtain the temporal CL loss.

MoCo consists of a trainable base encoder (θ_{base}) and a momentum encoder (θ_{mom}) updated via exponential moving average (EMA). While both encoders share identical architectures, the momentum encoder maintains slowly-evolving parameters through:

$$\theta_{\text{mom}} \leftarrow m\cdot\theta_{\text{mom}} + (1-m)\cdot\theta_{\text{base}} \quad (m\in[0,1)) \tag{4}$$

The anchor samples are processed by the base encoder, while the positive samples are encoded by the momentum encoder. This asymmetric design builds a dynamic queue through the stable feature space constructed by the momentum encoder, where the positive samples of the dictionary remain consistent.

Both encoder consist of a series of fully connected layers and dilated convolutional operations, where the fully connected layers reduce the dimensionality while retaining important features, improving efficiency and generalization. Dilated convolutions expand the receptive field, lowering computational costs without reducing the feature map size, thus minimizing information loss from the fully connected layers. After the first step of processing the network flow data through their respective encoders, the anchor matrix D_A^T and the positive matrix D_P^T are obtained, both with dimensions (T, K, D_o). Here, K represents the number of network flows, each flow is represented by D_o features, and each flow has undergone T transformations. This is immediately followed by 1D convolutions with varying kernel sizes to capture multi-granularity features, while achieving feature compression. The compressed dimension is (T, K, O), where $O = \lfloor \frac{D_o}{2} \rfloor + 1$. Within each iteration, one sample is randomly selected from the K samples in the anchor matrix to serve as the new anchor sample $A_t = [a_{t_1}, a_{t_2}, \ldots, a_{t_T}]^T \in \mathbb{R}^{T\times O}$, and one sample is randomly selected from the positive matrix to serve as the positive sample $P_t = [p_{t_1}, p_{t_2}, \ldots, p_{t_T}]^T \in \mathbb{R}^{T\times O}$. The contrastive loss is then computed between A_t and P_t.

The negative sample matrix $N_t = [n_{t_1}, n_{t_2}, \ldots, n_{t_{N_{neg}}}] \in \mathbb{R}^{D_o\times N_{neg}}$ is initialized with noise vectors drawn from a Gaussian distribution [2], where N_{neg} is a predefined hyperparameter denoting the number of negative samples. The noise vector is

initialized using a Gaussian distribution and embedded into the CL framework as a continuous and differentiable representation.

Consider an anchor sample a_{t_i}, and assume there exists a single positive sample p_{t_i} paired with a_{t_i}. The contrastive loss function operates on the principle that the loss should be low when a_{t_i} is similar to its positive pair p_{t_i} and dissimilar to all negative samples. We adopt the InfoNCE loss $\mathcal{L}_t$, defined as:

$$\mathcal{L}_t = -\frac{1}{T}\sum_{i=1}^{T}\log\frac{\exp\left(\frac{a_{t_i}\cdot p_{t_i}}{\tau}\right)}{\exp\left(\frac{a_{t_i}\cdot p_{t_i}}{\tau}\right)+\sum_{j=1}^{N_{neg}}\exp\left(\frac{a_{t_i}\cdot n_{t_j}}{\tau}\right)} \quad (5)$$

where $a_{t_i}\cdot p_{t_i}$ and $a_{t_i}\cdot n_{t_j}$ denotes the vector dot product (equivalent to cosine similarity for normalized vectors), and τ is the temperature hyperparameter.

C. Global Frequency-Based Contrastive Network

In this branch, supervised CL is performed. First, define the background matrix $\mathcal{B}$ as the matrix containing the representations of all samples in the current batch extracted by the encoder. This matrix serves as the foundational storage structure for sample representations, from which anchor samples and their corresponding positive and negative sample pairs can be constructed. Following the process described in Section B, we obtain D_A^T and D_P^T as the background matrices.

In the subsequent analysis, a discrete Fourier transform (DFT) is performed along the sample dimension (dim $=1$) on the feature matrix $\mathcal{B}\in\mathbb{R}^{T\times K\times D_o}$, converting the time-domain distribution of each feature into a frequency-domain representation. For the sequence $\mathcal{B}_{T_i,:,D_{o,i}} = [F_{T_i,0,D_{o,i}}, F_{T_i,1,D_{o,i}},\ldots,F_{T_i,K-1,D_{o,i}}]$, $T_i\in\{0,1,\ldots,T-1\}, D_{o,i}\in\{0,1,\ldots,D_o-1\}$ at T_i-th transformation and $D_{o,i}$-th feature across all flows, its DFT is:

$$F_{T_i,D_{o,i}}[f_k] = \sum_{n=0}^{K-1}\mathcal{B}_{T_i,n,D_{o,i}}\cdot e^{-j\frac{2\pi f_k n}{K}}, \quad f_k\in 0,1,\ldots,\left\lfloor\frac{K}{2}\right\rfloor+1 \quad (6)$$

$F_{T_i,D_{o,i}}[f_k]$ represents the f_k-th frequency component of the DFT result for the $D_{o,i}$-th feature in the T_i-th transformation. Since the Fourier transform of a real-valued signal is conjugate symmetric, only the positive frequency components are returned. For all T iterations of data augmentation, each iteration computes the frequency-domain representation of D_o features distributed across all K network flows, with the output result being $\tilde{\mathcal{B}}\in\mathbb{R}^{(T\times F_K\times D_o)}$, where $F_K = \lfloor\frac{K}{2}\rfloor+1$.

To enhance the representation learning in the Fourier layer and promote interactions among different frequency components, we introduce an affine transformation composed of a linear transformation and an offset vector. For each transformation T_i and frequency component $f_k\in\{0,1,\ldots,F_K\}$, multiply the D_o-dimensional feature vector at that frequency by weight matrix $W_{f_k,D_{o,i},O_i}(O_i\in\{1,2,\ldots,O-1\})$, then add bias term B_{f_k,O_i} to obtain $\mathcal{B}_L\in\mathbb{R}^{T\times F_K\times O}$. The subscript of each matrix in the following formula indicates the dimension.

$$\mathcal{B}_L = \sum_{D_{o,i}=0}^{D_o-1}\tilde{\mathcal{B}}_{T_i,f_k,D_{o,i}}\cdot W_{f_k,D_{o,i},O_i} + B_{f_k,O_i} \quad (7)$$

The DFT decomposes network traffic features into distinct frequency components, where each frequency corresponds to a specific periodicity. An affine transformation assigns weights and biases to each frequency component f_k, automatically amplifying or diminishing the influence of current features.

Next, reconstruct the time-domain network flow sequence from $\mathcal{B}_L$ by performing the inverse DFT (IDFT) along dim=1, obtaining the reconstructed sequence. Previously, we extracted inter-flow periodic patterns by applying the DFT along dim=1 (across different flows). Now, to capture intra-flow periodicity within individual flows, we will similarly transform the reconstructed sequences into the frequency domain by performing DFT along dim=2. The resulting frequency-domain representation $\tilde{\mathcal{B}_L}\in\mathbb{R}^{T\times K\times O}$ will enable multi-scale frequency feature extraction, where each element $B_{T_i,K_i,O_i}(K_i\in\{0,1,\ldots,K-1\}$ is a complex number, denoted as $B_{T_i,K_i,O_i} = \mathrm{Re}_{T_i,K_i,O_i} + j\cdot\mathrm{Im}_{T_i,K_i,O_i}$. In the following work, we will only consider the amplitude and obtain:

$$B_A(T_i,K_i,O_i) = \sqrt{(\mathrm{Re}_{T_i,K_i,O_i})^2+(\mathrm{Im}_{T_i,K_i,O_i})^2} \quad (8)$$

Consequently, the amplitude matrix is obtained by computing the complex modulus of all elements in matrix $\tilde{\mathcal{B}_L}$. We randomly select a complete data set from the T transformations of the amplitude matrix to construct $\mathcal{B}_A\in\mathbb{R}^{K\times O}$. The contrastive loss is calculated by measuring feature similarity between positive and negative sample pairs within $\mathcal{B}_A$. Positive samples refer to sample pairs belonging to the same class, while negative samples represent pairs from different classes. Similarity is measured by the dot product between samples, where higher values indicate greater similarity.

Experimental observations show that most similarity values exceed 0.5, suggesting that instances classified as negative samples exhibit high similarity. This implies that the boundary between different classes is relatively ambiguous. The similarity matrices for negative pairs are denoted as N_f. Therefore, a stronger penalty is applied only to negative sample pairs. The new loss function with a relaxation factor is formulated as follows:

$$\mathcal{L}_f = \log\left(\sum_{i=1}^{N_{neg}} e^{g(N_f)\lambda\gamma}\right) \quad (9)$$

where N_{neg} represents the number of pairs of negative samples in a certain dataset. We determine the value of the hyperparameter γ by experiments. $g(N_f)\gamma$ achieves a stricter data separation through the hard margin, providing a closer approximation to

TABLE III
DATASET DETAILS

Dataset	Number of Categories	Benign/Attack Type	Total Num. of samples	Ratio of Benign to Malicious
MQTT-IoT-IDS2020 (MQTT)	5	Benign, Aggressive scan, UDP scan, Sparta SSH brute-force, MQTT brute-force	262,301	3:2
UNSW-NB15	10	Benign, Analysis, Backdoor, DoS, Exploits, Fuzzers,Generic, Reconnaissance, Shellcode, Worms	2,540,044	7:1
USTC-TFC2016	11	Benign, Cridex (Dridex), Geodo (Emotet), Htbot, Miuref, Neris, Nsis-a, Shifu, Tinba, Virut, Zeus	7,128,610	3:2
CIC-DDoS2019	13	Benign, DNS, LDAP, MSSQL, NetBIOS, NTP, SNMP, SSDP, SYN, TFTP, UDP, UDP-Lag, WebDDoS	50,063,112	1:879
TON-IoT	10	Benign, Injection, MITM, Backdoor, DDoS, DoS, Ransomware, Scanning, XSS, Password	16,940,523	9:25

The first three columns in this table are used in our actual experiments, while the last two columns represent the information provided by the official dataset. For specific details on the experimental data partition, refer to Fig. 4.

representing the data. $g(\cdot)$ represents the relaxation function as:

$$\lambda = N_f + m$$
$$g(N_f) = \max\{0, N_f - m\}$$
$$m \leq 0.5 \tag{10}$$

It is worth mentioning that, we did not strictly differentiate the inner product similarity between (non-)similar data as 0 or 1. Instead, we used m, which is a relaxation boundary between samples of different classes. The adoption of relaxation factors allows the model to pay more attention to challenging samples during learning, thereby avoiding overfitting on the negative samples with low similarity itself.

V. EXPERIMENTS

A. Experimental Setup

The experiments were conducted within the Ubuntu 20.04 operating system, employing an NVIDIA GeForce RTX 2080 Ti, accompanied by six Intel(R) Xeon(R) Silver 4216 CPU @ 2.10 GHz processors, and a total of 251 gigabytes of memory. The experimentation environment was configured with Python 3.8.16, CUDA 12.2, and PyTorch 1.9.0+cu102.

Datasets and Settings: We evaluate the proposed method on five publicly available datasets, and more details about the dataset will be presented in Table III. The datasets used for comparison include MQTT-IoT-IDS2020 [4], UNSW-NB15 [7], USTC-TFC2016 [3], CIC-DDoS2019 [17], and TON-IoT [18]. They offer diverse scenarios, from simulated IoT networks to real network testing platforms, covering various attack types such as DDoS attacks and content-based attacks like injection and XSS.

We utilized pcap-formatted data for each dataset and applied the USTC-TK2016[1] tool uniformly. The process involved converting pcap data into decimal format. Each network flow consisted of 28 data packets, each capturing the first 28 B, resulting in a 28∗28 decimal array for each flow. Additionally, we designed five distinct tasks, denoted as T_1, T_2, T_3, T_4 and T_5.

[1]https://github.com/yungshenglu/USTC-TK2016

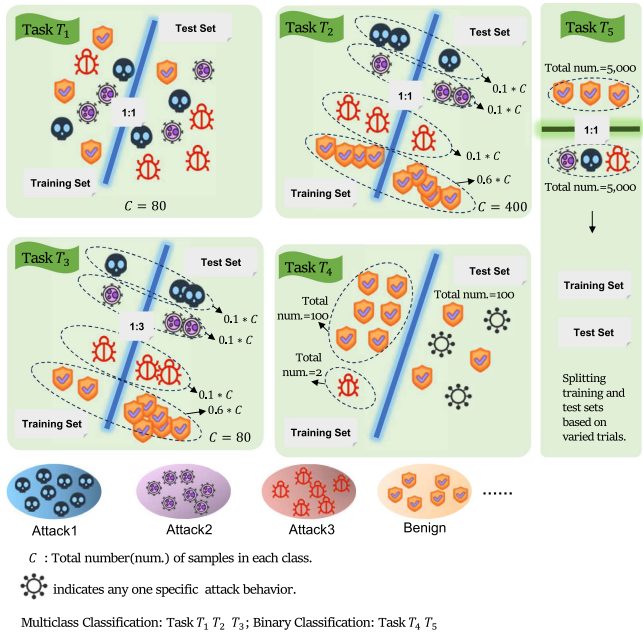


Fig. 4. Data distribution for five tasks (T_1, T_2, T_3, T_4 and T_5).

Each task exhibits increasing levels of difficulty. T_1 represents a multiclass task with a balanced dataset. T_2 represents an imbalanced training dataset, where the normal class accounts for 0.6 of the total instances in the normal class, and each category of malware accounts for 0.1 of its respective category's total instances. T_3 has led to an increase in the complexity of the T_2, which in turn resulted in a reduction in the size of the training set and an increase in the proportion of the test set. T_4 is devised as a binary classification task with an imbalanced dataset. In T_5, switched to a larger dataset, with 5000 instances each for benign data and attack behaviors. For specific partitioning details, please refer to Fig. 4.

To clarify, we'll use task T_2 as an illustrative example in Fig. 4. When $C = 400$, the number of normal samples in the training set is $0.6 * C = 240$. This sample size is not considered large for easily obtainable normal traffic. The number of samples for other malware categories is $0.1 * C = 40$ each.

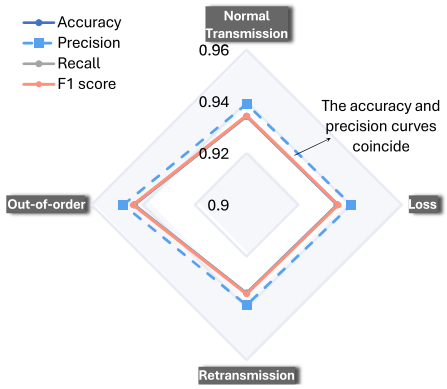


Fig. 5. Validation of the selection of network induction environment.

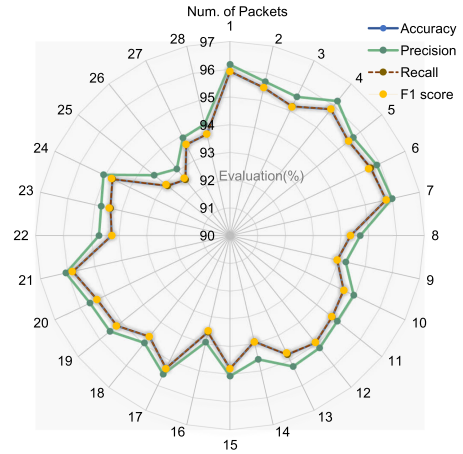


Fig. 6. Validation of the impact of packet quantity on detection results.

To mitigate the effects of randomness, we averaged the results of each experiment over ten different random seeds. The main hyperparameters that affect the computation of $\mathcal{L}_f$ are the initialization of γ, as well as the adjustment process during backpropagation. We directly set γ to $6.4 * 10^6$. The reason for this configuration was determined through experimentation, as detailed in the second part of trial D.

For the parameters of the detection head, the settings are as follows. In the XGBoost model, the number of trees is 10, the step size is 0.1, and the maximum depth of each tree is 6; in the RF and ET, the values of the important parameters are the same, the number of trees is 100, and the splitting criterion is based on the Gini coefficient. The minimum number of samples that a node can split is 2, and the minimum number of samples required for a leaf node is 1.

B. Preliminary Parameters

Before initiating the experiments, we posed three questions to address uncertainties regarding the creation and preprocessing of input network flows:

Q1: Which network induction environment yields better detection results?

Q2: How effective is single-packet detection?

Q3:Is the detection effective for content-based attack behaviors?

1) Solution of Q1: The network flows are primarily enhanced by simulating common network-induced phenomena, including normal transmission, retransmission, loss, and out-of-order, to improve the robustness of the model. This trial will verify which induced phenomenon yields the best detection results. For this purpose, we use the CIC-DDoS2019 dataset under the data distribution of T_2, and detailed results are shown in Fig. 5.

The results indicate that the detection outcomes for normal transmission, retransmission, and loss inductions are similar, while the results for out-of-order are slightly higher than the others. This phenomenon can be approximated as a combination of multiple random noise processes designed to confuse the original sequence, enhancing the model's robustness. Therefore, in the subsequent experiments, we will use out-of-order as the pre-processing method for the network traffic.

2) Solution of Q2: We set the number of packets in a network flow to be 28. However, with a larger number of packets, it implies that a certain amount of packets needs to be accumulated in advance for detection, inevitably causing significant delays in real-time packet detection. Therefore, the first step is to validate to what extent the number of packets can impact the detection results. To address this question, we partitioned the training and test sets on the USTC-TFC2016 dataset according to the distribution of T_2. Starting with one data packet, we incrementally added one data packet at a time, checking the test results after each length change. The experimental results are shown in Fig. 6.

Observing the results depicted in Fig. 6, an interesting finding emerged. Surprisingly, the detection performance was notably effective when considering only a single packet. However, as the number of packets increased, especially beyond the 21th packet, the test results started to decline. This decline might be attributed to subsequent packets being empty, and for network flows with fewer than 28 packets, padding with zeros led to meaningless data in the latter part of the sequence.

Consequently, MTRF demonstrates the capability to effectively detect malicious behavior even with a limited number of packets, potentially as few as one. The choice of 28 packets was made to accommodate the general scenarios present in most datasets.

To conduct an in-depth analysis of the information captured by individual data packets, the mutual information (Mutual Information, MI) was calculated between each data packet feature value and the label. MI quantifies the statistical dependence relationship between the feature and the label, and the higher the value, the more effective the feature is for classification.For the core fields of the IP layer(MI=0.4111), such as TOS, protocol field, and flags are universal features for traffic classification. Link-layer fields, including Ethernet type(MI=0.4081) and MAC address(MI=0.4085), are also of secondary importance. Since the USTC-TFC2016 dataset was collected using different virtual machine simulations to generate traffic, the MAC addresses also play a certain role in classification.

TABLE IV
RESULTS OF ANOMALY DETECTION ON PACKET PAYLOADS

	Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score
	First 28 packets with the initial 28 bytes				First 28 packets with bytes 28 to 100 in the middle			
Only Injection	0.9591	0.9641	0.9591	0.9598	0.9994	0.9994	0.9994	0.9994
Only XSS	0.9683	0.9713	0.9683	0.9687	0.9995	0.9995	0.9995	0.9995
XSS+Injection	0.9734	0.9752	0.9734	0.9736	0.9994	0.9994	0.9994	0.9994
ALL	0.9953	0.9953	0.9953	0.9953	0.9210	0.9235	0.9210	0.9206

TABLE V
COMPARE THE PERFORMANCE METRICS OF DIFFERENT METHODS ON THE UNSW-NB15 DATASET FOR THE T_1

Method	Accuracy	Precision	Recall	F1 score	Training time(s) (until Convergence)	Testing time/ per sample(ms)
Trans-learn [14]	0.2035	0.1100	0.2000	0.1300	35.82	890.00
TDA [10]	0.2128	0.3557	0.2128	0.2553	2.73	0.55
KSWIN [11]	0.3725	0.4257	0.3725	0.3739	95.72	49720.90
Mal-CNN [15]	0.5100	0.5114	0.5541	0.4998	8.10	500.00
ERNN [1]	0.5800	0.6545	0.5792	0.5692	8.33	7.00
E-GraphSAGE [8]	0.6232	0.6678	0.6790	0.6326	18.24	5.70
Dyn-rep [16]	0.6552	0.6497	0.6552	0.6508	10.06	4.20
GD-DBLR [13]	0.7000	0.7037	0.7000	0.7001	18.58	0.06
Deep-Packet [17]	0.7671	0.7581	0.7671	0.7529	18.99	4450.00
MTH-IDS [12]	0.7674	0.7915	0.7574	0.7645	20.00	257.00
ET	0.5700	0.5611	0.5700	0.5517	-	-
RF	0.5750	0.5567	0.5750	0.5541	-	-
DT	0.5850	0.5850	0.5932	0.5850	-	-
XGBoost	0.6300	0.6129	0.6300	0.6142	-	-
MTRF(Ours)-XGBoost	0.9194	0.9353	0.9194	0.9175	37.33	23.90+0.04
MTRF(Ours)-DT	0.9203	0.9365	0.9203	0.9182		
MTRF(Ours)-RF	0.9633	0.9654	0.9633	0.9633		
MTRF(Ours)-ET	**0.9699**	**0.9714**	**0.9699**	**0.9699**		

The models marked in bold indicate the top-performing ones within each category, while models marked with underlines represent the second-best performers.

3) Solution of Q3: The question stems from extracting the initial 28 bytes of each packet, which includes a substantial portion of packet header information. We need to verify the effectiveness of detecting payload-based malware implantation. Injection attacks like SQL injection and XSS achieve their goals by injecting malicious code into an application's content. Hence, we conduct experiments using the TON-IoT dataset (including injection attacks, as shown in Table III). Additionally, we extract bytes 28-100 of each packet to include maximum payload information while excluding headers. Following the setup of T_5, we allocate 100 samples for the training set and 9,900 samples for the test set. The experimental results are presented in Table IV.

Compared to extracting payload information (with an accuracy rate of 0.9994), using packet header extraction does not perform well in detecting injection attacks and XSS attacks (with an accuracy rate of 0.9734). However, when more attack behaviors are mixed in, the opposite is true. The detection of extracting the header information becomes more effective (0.9953 vs. 0.9210). This suggests that our model is effective in handling both headers and payloads. The key lies in understanding the general categories of such attack behaviors before segmenting and extracting packet data, significantly enhancing detection accuracy.

C. Experimental Results

1) Following the Settings of Task T_1: We conducted experiments on the UNSW-NB15 dataset and combined the data distribution from T_1 to verify the intrusion detection capability of our MTRF model. Various competition models are compared. We selected (non-)end-to-end models from related works for competitive analysis. Here, we process each model according to its original form of input samples. For instance, [8] utilizes IP addresses and ports to represent nodes, so we extracted these two features to represent nodes in the graph. However, the sample sizes of the training and testing sets used are consistent with those we set. For the four ML models, including ET, RF, DT, and XGBoost, the input data is identical to that of the MTRF model. The purpose is to compare the representational capability of MTRF to what extent it improves over directly inputting raw data, while keeping the detection models consistent.

In Table V, the data in the last four rows were represented using the proposed models, followed by the application of four different classifiers for detection. While both RF and ET models performed similarly, ET exhibited the best performance with an F1-score of 0.9699. Table V also lists the training convergence time and average inference time per sample for each model. MTRF represents each test sample (took about 23.90 ms) before feeding it into the ML model for testing (took about 0.04 ms),

TABLE VI
COMPARE THE PERFORMANCE METRICS OF VARIOUS METHODS ON THE UNSW-NB15 DATASET FOR TASKS T_2 AND T_3, AND ON THE MQTT DATASET FOR TASK T_4

Model / Task	T_2	T_3	T_4
	Accuracy / Precision / Recall / F1-score		
RF [12]	0.5350 / 0.5349 / 0.5330 / 0.5209	0.3800 / 0.3927 / 0.3800 / 0.3772	0.8900 / 0.8604 / 0.7440 / 0.7400
ET [12]	0.5350 / 0.4845 / 0.4879 / 0.4717	0.4050 / 0.3971 / 0.4050 / 0.3897	0.8900 / 0.8880 / 0.8290 / 0.8277
XGBoost [12]	0.5350 / 0.5459 / 0.5414 / 0.5352	0.4472 / 0.4694 / 0.4722 / 0.4633	0.8800 / 0.1024 / 0.3199 / 0.1551
DT [12]	0.5325 / 0.5407 / 0.5325 / 0.5345	0.5100 / 0.4841 / 0.5100 / 0.4851	0.7570 / 0.8678 / 0.7570 / 0.7520
MTRF(Ours)-ET	**0.9507 / 0.9534 / 0.9507 / 0.9506**	**0.8486 / 0.8591 / 0.8486 / 0.8496**	**0.9500 / 0.9543 / 0.9500 / 0.9494**

The models marked in bold indicate the top-performing ones within each category, while models marked with underlines represent the second-best performers.

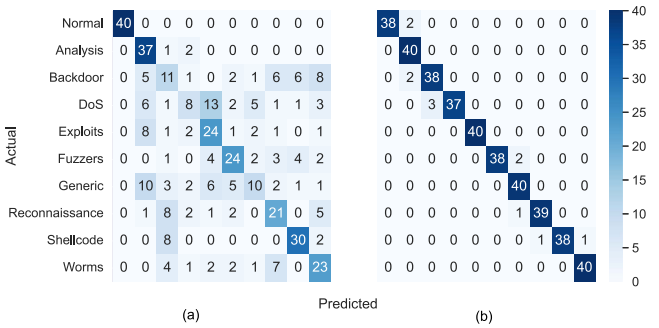


Fig. 7. The heat map for intrusion detection with (a) the ET model and (b) the MTRF-ET model under the task T_1.

resulting in a total inference time of 23.94 ms per sample. It is observed that MTRF achieves the best detection performance, although its training and testing times are at a moderate level.

To provide a more detailed representation of the improvement achieved by MTRF-ET over the ET model, we visualize the performance results using heat map. As shown in Fig. 7, each element represents the number of samples that match the actual and predicted class. Among them, (a) is the prediction result of the ET model, and (b) is the prediction result of the MTRF-ET model. From Fig. 7, we observe that in (b), the poorest classification performance is for DoS attacks, with only three samples being misclassified. In (a), the worst-performing class is also DoS, with its misclassified instances spread across the other eight classes.

All competing models adhere to their original input requirements. With T_1's distribution containing 80 samples per category, the training and test sets are divided equally. This results in the training set having only 400 samples that come from the UNSW-NB15 dataset. In contrast, E-GraphSAGE requires datasets in the tens of millions for multi-class detection. The limited dataset size is the primary reason why most competing models struggle to accurately classify the 10 categories. Unlike other methods that rely on large-scale datasets for training, MTRF offers an effective data representation approach without imposing specific requirements on the classification network. Furthermore, our approach requires extracting only a few packets from each flow, eliminating the need for additional feature extraction.

2) Following the Settings of T_2 and T_3: In this experiment, we will continue conducting imbalanced 10-class classification tests using the UNSW-NB15 dataset. Based on our findings in the T_1 task, we observed that the ET classifier yielded the best detection performance. Therefore, we will continue to apply this technique in the T_2 and T_3 tasks. Based on the observations from Table VI, it can be observed that the performance of all models has declined to varying degrees compared to T_1. However, in these two tasks, MTRF consistently maintains a significant lead. Competitors evidently struggle to adapt to the challenges of multi-class classification. Because the highest F1 score in both tasks is nearly twice as high as the competitors'highest score.

3) Following the Settings of T_4: In the upcoming experiments, we will conduct the T_4 task on the MQTT dataset, which has fewer categories. The experimental results are as shown in Table VI. In this task, MTRF's performance closely matches that in the T_2 task. Other competitors have shown significant improvements, with a preference for binary classification scenarios. Surprisingly, DT performs unexpectedly poorly, exhibiting minimal effectiveness in this context.

Taking into account the experimental results of T_2, T_3 and T_4, it can be concluded that MTRF is suitable for imbalanced datasets in both multi-class and binary classification scenarios.

4) Following the Settings of T_5: In all previous experiments, only a small number of samples were used for training. Therefore, this trial will increase the data volume to further validate the effectiveness of MTRF on a larger training set. In this trial, we employed a balanced binary detection. There were 5000 samples for both benign and malicious flows, while all malicious behaviors are assigned one label.

We conducted experiments on the USTC-TFC2016 dataset, and the results are shown in Fig. 8. The numbers below each subplot indicate the actual ratio of samples in the training set to the test set. This series of results indicate that as the ratio of the training set to the test set changes, the inference results vary minimally, remaining within the range of 0.980 to 0.985. This suggests that MTRF exhibits good stability in balanced binary classification.

D. Further Analysis

1) Analysis on Domain Transformation: This trial aims to investigate the influence of temporal-domain and frequency-domain transformations on the contrastive loss. We conducted

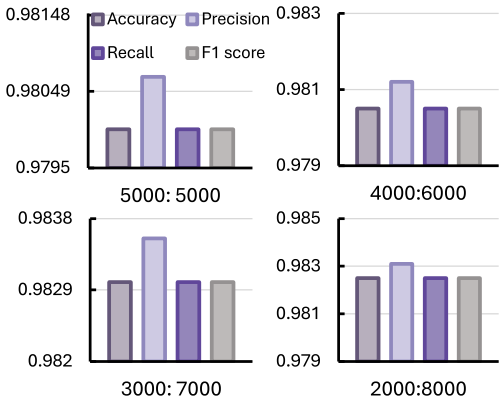


Fig. 8. The total number of samples for both the training and testing sets was expanded to 10,000 in a binary classification conducted on the USTC-TFC2016 dataset.

TABLE VII
ABLATION EXPERIMENTS ON DOMAIN TRANSFORMATIONS CONDUCTED ON THE MQTT DATASET FOR T_1 AND T_2 TASKS

Domain / Task		T_1			
$\mathcal{L}_t$	$\mathcal{L}_f$	**Accuracy**	**Precision**	**Recall**	**F1 score**
✓		0.9849	0.9857	0.9849	0.9849
	✓	0.9839	0.9843	0.9839	0.9839
✓	✓	0.9879	0.9884	0.9879	0.9879
Domain / Task		T_2			
✓		0.9692	0.9706	0.9692	0.9693
	✓	0.9747	0.9757	0.9747	0.9747
✓	✓	0.9802	0.9809	0.9802	0.9802

ablation experiments to investigate their impact on MTRF. This trial used the MQTT dataset following the data distribution of T_1 and T_2. The results are presented in Table VII.

Our experiments show that solely using temporal-domain features ($\mathcal{L}t$) or frequency-domain features ($\mathcal{L}f$) exhibits detection capabilities for different-distribution datasets. However, the fusion of data after undergoing multiple domain transformations yields superior results.

2) Analysis on Hard Margin: We also investigated the impact of the hard margin parameter γ within the frequency domain loss function (9) on intrusion detection. The results are shown in Fig. 9, where the horizontal axis represents the numerical variations of the parameter γ ranging from $6.4 * 10^2$ to $6.4 * 10^7$, increasing by a factor of 10.As γ increases, all performance metrics improve, especially rapidly initially. This indicates a stronger demand for γ growth in intrusion detection performance. However, it is worth noting that a larger γ is not necessarily better. Once the parameter exceeds 6.4×10^6, further increases lead to performance degradation.

3) Analysis on Representation: As a representation model, MTRF should generalize learned data representations to unseen test data. To clarify the meaning of the unknown test dataset, we define the following symbols. The network flow categories are denoted by F. F_{tr} and F_{te} respectively signify the categories present in the training and testing sets. The sets of samples within

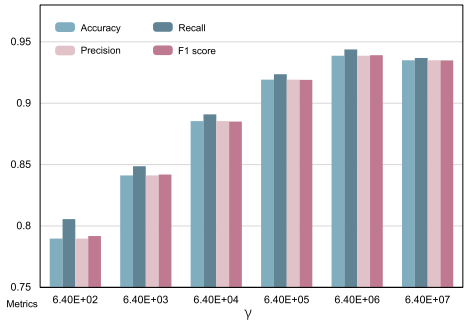


Fig. 9. Compare the classification results by varying the hard margin following the distribution of the training data in T_3 on the MQTT dataset.

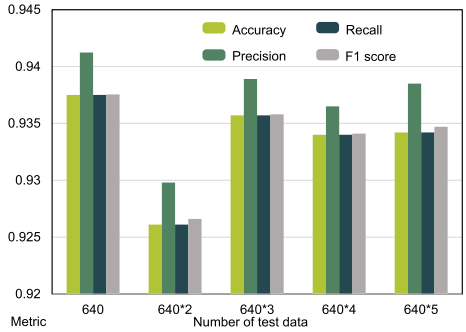


Fig. 10. In the USTC-TFC2016 dataset with T_2 distribution, we gradually increased the size of the test set to obtain intrusion detection performance.

the training and testing datasets are succinctly represented as X_{tr} and X_{te}. *Definition 1:* The presence of unknown samples in the test set, symbolically expressed as $F_{tr} = F_{te}$, $X_{tr} \neq X_{te}$. *Definition 2:* The occurrence of unknown categories in the test set, symbolically expressed as $F_{tr} \neq F_{te}$, $X_{tr} \neq X_{te}$. Therefore, we designed two experiments, namely data augmentation and dataset transfer, from these two perspectives to demonstrate the representation capabilities of MTRF.

Data Augmentation: To verify MTRF's generalizability to unseen data, we progressively expanded the test set while preserving the training data distribution of T_2 in the USTC-TFC2016 dataset. In accordance with *Definition 1*, only unknown samples are added to the test set. The ultimate exploration yields the classification results for 11 types of network behaviors on this dataset.

The experimental results are shown in Fig. 10, where the horizontal axis represents the size of the test data, growing as a multiple. From the Fig. 10, it can be observed that the performance of intrusion detection does not exhibit a significant decline as the size of the test set increases. This shows MTRF's strong representation ability, effectively learning general patterns from training data and generalizing well to unseen datasets.

Dataset Transferability: To assess the generalization ability of the MTRF model, we aim to determine its capacity to represent previously unseen datasets aligning with *Definition 2*. Specifically, MTRF learns parameters from known datasets, then directly applies them to represent and detect intrusions in

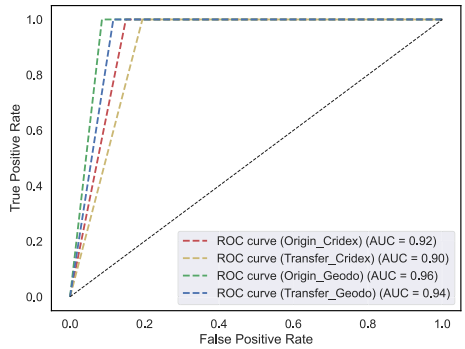


Fig. 11. The ROC curve obtained by directly representing the data from the USTC-TK2016 dataset using the MTRF model trained on the MQTT dataset with data distribution task T_4.

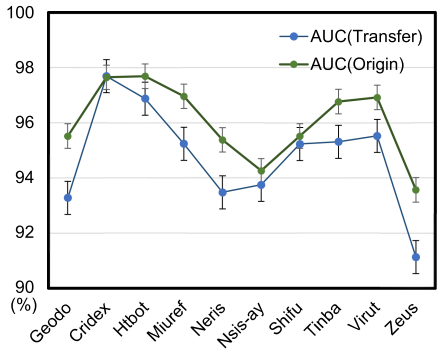


Fig. 12. The model trained on the CICDDoS-2019 dataset is transferred for validation on the USTC-TK2016 dataset.

unseen data. Check whether its performance is comparable to that achieved by training solely based on unseen data.

To achieve this, we initially train the MTRF model on the MQTT dataset and saved the model parameters. Then, these parameters were loaded to represent the previously unseen USTC-TK2016 dataset. We utilize ROC curves to observe intrusion detection performance, helping us ascertain whether such representations remain effective.

The specific results are shown in Fig. 11. We conducted binary classification using the T_4 data distribution. We tested the intrusion detection results of the malware Cridex and Geodo in the USTC-TK2016 dataset, denoted as Origin_Cridex and Origi_Geodo, respectively. After reloading the model, we tested the classification results for the same two types of malware, denoted as Transfer_Cridex and Transfer_Geodo. From Fig. 11, it can be observed that the results after model loading are 0.90 and 0.94, slightly lower compared to 0.92 and 0.96. This suggests that MTRF exhibits a certain degree of generalization capability in terms of representation.

In-depth Analysis of Dataset Transferability: In MQTT data, the observed attack behaviors primarily involve brute-force attacks or scanning attacks utilizing Geodo (a botnet). These attack behaviors bear resemblance to trojans and worm viruses found in the USTC-TFC2016 dataset. Therefore, we need to conduct more comprehensive experiments to thoroughly validate the transferability of the dataset.

Following T_4 data distribution, we trained a model on the CICDDoS-2019 dataset for malicious traffic detection. The model was then directly applied to the USTC-TK2016 dataset, evaluating its performance on each category using AUC as the detection metric. The experimental results are illustrated in Fig. 12.

The CICDDoS-2019 dataset primarily focuses on protocol-specific attacks, while the USTC-TK2016 dataset concentrates on trojans and worm viruses. Therefore, the similarity between the attack types of the two datasets is relatively low. However, observing the AUC in the graph, there is only a minor decrease of around 2%. In fact, under the T_4 data distribution, the training of the transferable model only utilized two attack samples. We speculate that this might be due to the strong representation capabilities for benign samples. Therefore, visualizations were conducted on the samples before and after representation.

Our specific approach is as follows: T_4 proportions were used to mix DNS attack behavior and benign traffic from the CICDDoS2019 dataset. The trained model was then directly applied to the MQTT dataset and the USTC-TK2016 dataset, obtaining characterized samples. The x-axis represents the flattened one-dimensional representation of each sample. For instance, in (a), after characterization, each sample is represented as a (1,320)-dimensional vector, resulting in 320 values along the x-axis.The results are illustrated in Fig. 13.

Initially chaotic data distributions (Fig. 13(b)(c)) become clearly separable between benign and malicious flows after representation, demonstrating MTRF's effectiveness. Additionally, the data distribution of benign flows after representation becomes more structured.

4) Large-Scale Imbalanced Dataset: The false positive (FP) and false negative (FN) are crucial assessment metrics for reliable intrusion detection. A high FP trigger unnecessary false alarms, while a high FN leaves threats undetected. We conducted binary classification experiments on the TON-IoT dataset, comprising 9500 benign samples and 500 attack samples (featuring various attack behaviors). The training and testing sets were sized at 100 and 9900, respectively, and the experimental results are depicted in Fig. 14.

According to Fig. 14, it can be observed that the MTRF-ET model yields FP $=$ 0 and FN $=$ 74 in its detections, while using only the ET model results in FP $=$ 63 and FN $=$ 404. This indicates that the MTRF demonstrates effective detection performance on imbalanced datasets. The low FP rate helps reduce false alarms, providing benefits for maintaining system stability and reducing business interruptions.

Statistical Significance Tests: We selected the dataset CICDDoS2019 which has 13 categories, and for each category, we chose 100 samples. By applying the DFT to each sample's feature dimensions, the amplitude spectra are obtained. The frequency domain feature changes before and after the MTRF model representation are compared in detail. The study visually presents the differences in the amplitude spectrum distribution before and after the representation. Mltivariate analysis of variance (MANOVA) was further employed to evaluate the

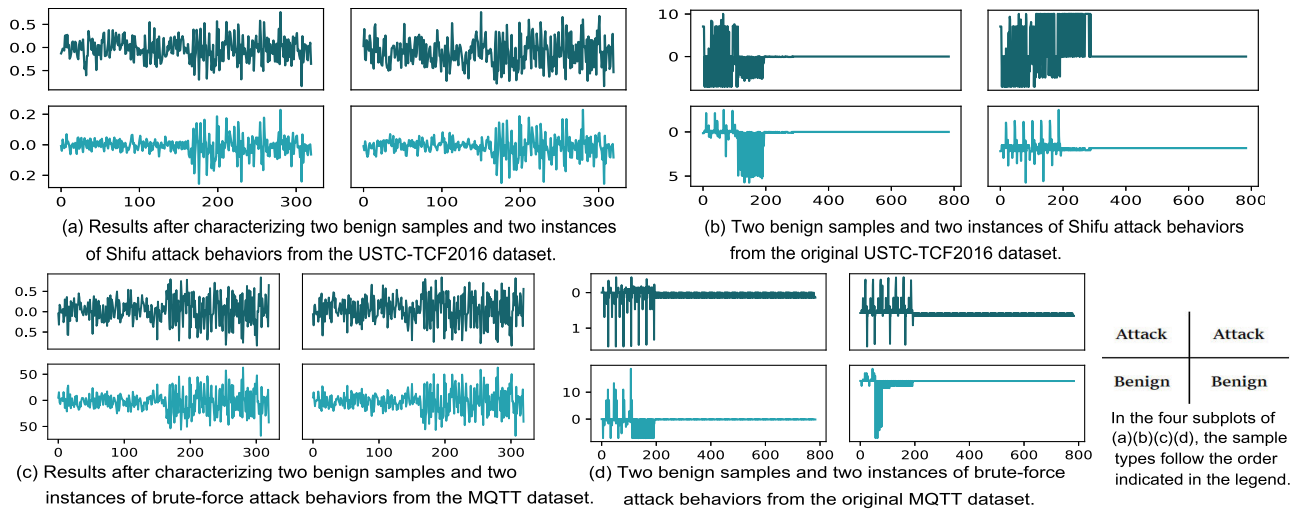


Fig. 13. Visualization of network flow samples before and after representation.

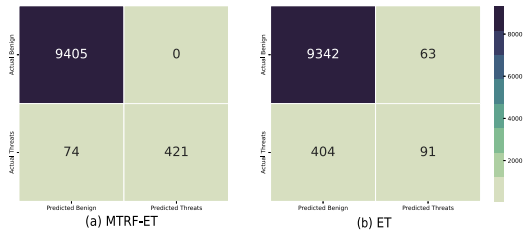


Fig. 14. The impact of MTRF on FP and FN in imbalanced datasets.

Fig. 15. Compare the amplitude spectra before and after using MTRF for representation.

multi-class discrimination enhancement of post-representation amplitude spectra. Fig. 15. shows the amplitude spectrum, with the horizontal axis representing the normalized frequency ranging from 0 to 0.5. It can be seen that the 13 types of network flows can be clearly distinguished in the amplitude spectra after the representation.

Table VIII shows the results of MANOVA. It can be seen that the network flow characterized by MTRF has an F value as high as 163.6 (5.7 times that of the original), the Wilks' lambda

TABLE VIII
THE RESULTS OF MANOVA

	Value	Num DF	Den DF	F Value
Original Data				
Wilks's lambda ↓	0.0754	388.0000	912.0000	28.8362
Pillai's trace ↑	0.9246	388.0000	912.0000	28.8364
Hotelling-Lawley trace ↑	12.2680	388.0000	912.0000	28.8362
Roy's greatest root ↑	12.2680	388.0000	912.0000	28.8362
Data Represented by MTRF				
Wilks's lambda	0.0415	161.0000	1139.0000	163.5959
Pillai's trace	0.9585	161.0000	1139.0000	163.5959
Hotelling-Lawley trace	23.1246	161.0000	1139.0000	163.5959
Roy's greatest root	23.1246	161.0000	1139.0000	163.5959

The direction of the arrow (↑ and ↓) indicates that the larger (smaller) the value, the better the performance.

value is smaller (0.0415 vs 0.0754), and the Hotelling-Lawley trace value is larger (23.12 vs 12.27). Moreover, the number of characteristic features of the characterized network flow is half of the original. This result proves that the feature set of MTRF has better discrimination efficiency and stronger class discrimination ability.

VI. RESOURCE CONSUMPTION

During the training phase, the resident set size (RES) of the MTRF model was approximately 3.038 GB, the shared memory size (SHR) was about 573.67 MB, the CPU requirement was approximately 13 cores, and the memory occupancy rate (%) was 1.2%. Therefore, the training task is suitable for being carried out on a cloud platform. After the training is completed, the model can be saved, and its size is only 150 MB.

When performing inference on the fog node device, while the occupation of other resources remains basically unchanged, the CPU usage approaches 2 cores. To prevent overloading, the fog node requires over 2 CPU cores and 4 GB of combined physical and shared memory. The disk space only needs to be sufficient for more than 150 MB.

In terms of real-time performance, the single inference time of the model is approximately 24 ms (refer to Table V). Moreover, since the fog nodes are close to the IoT devices and the communication delay is low, the overall system can meet the response requirements of most IoT real-time monitoring scenarios.

The scalability of the model mainly depends on the resource redundancy of the fog nodes. A single MTRF instance only requires approximately 2 CPU cores and 3.5 GB of memory. For fog node devices with multiple cores and large memory, multiple MTRF model instances can be deployed in parallel to enhance processing speed. Additionally, the model size is relatively small (150 MB), and it does not impose significant pressure on the storage resources of the fog nodes. If a fog node cluster is adopted for deployment, the detection tasks can be distributed to multiple nodes, thereby achieving good overall scalability.

VII. Conclusion

In our work, we separate the network flow representation learning from downstream classification tasks, avoiding the direct end-to-end learning and detection dependence on large-scale labeled samples. This has also been confirmed by comparing with previous state-of-the-art end-to-end anomaly detection methods. MTRF employs a hard margin strategy to represent packet data in the frequency domain, and it has been demonstrated that a larger parameter γ within a certain range can achieve better intrusion detection results. A large number of experiments have shown that our method outperforms fourteen other models in multi-class tasks, achieving the best performance. Furthermore, as a representation model, good generalization capability is crucial for MTRF. This is because it means that the model can be applied to unseen datasets. Further still, even if trained solely based on the existing dataset, it can still achieve outstanding detection performance. Even when training on completely unknown data using pre-loaded model parameters, MTRF can achieves AUC scores of 0.90 and 0.92. This is only 0.02 lower than the AUC obtained by training on the unknown data from scratch.

References

[1] Z. Zhao et al., "ERNN: Error-resilient RNN for encrypted traffic detection towards network-induced phenomena," *IEEE Trans. Dependable Secure Comput.*vol. 22, no. 1, pp. 493–510, Jan.-Feb. 2025.

[2] K. Ethayarajh, "How contextual are contextualized word representations? Comparing the geometry of BERT, ELMo, and GPT-2 embeddings.," in *Proc. Conf. Empirical Methods Natural Lang. Process.-Int. Joint Conf. Natural Lang. Process.*, 2019,pp. 55–65.

[3] W. W. Zhu et al., "Malware traffic classification using convolutional neural network for representation learning," *Proc. Int. Conf. Inf. Netw.*, 2017, pp. 712–717.

[4] H. Hindy et al., "Machine learning based IoT intrusion detection system: An MQTT case study (MQTT-IoT-IDS2020 dataset," in *Proc. Int. Netw. Conf.*, Cham: Springer International Publishing, 2020, pp. 73–84.

[5] G. Woo et al., "CoST: Contrastive learning of disentangled seasonal-trend representations for time series forecasting," in *Proc. Int. Conf. Learn. Representations*, Virtual Event, 2022, pp. 1–18.

[6] J. A. Barnes et al., "Characterization of frequency stability," *IEEE Trans. Instrum. Meas.*, vol. IM-20, no. 2, pp. 105–120, May 1971, doi: 10.1109/TIM.1971.5570702.

[7] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf.*, Canberra, ACT, Australia, 2015, pp. 1–6, doi: 10.1109/MilCIS.2015.7348942.

[8] W. W. Lo et al., "E-graphsage: A graph neural network based intrusion detection system for IoT," in *Proc. NOMS 2022-2022 IEEE/IFIP Netw. Oper. Manage. Symp. IEEE*, 2022, pp. 1–9.

[9] N. Wang, Y. Chen, Y. Hu, W. Lou, and Y. T. Hou, "FeCo: Boosting intrusion detection capability in IoT networks via contrastive learning," in *Proc. IEEE INFOCOM 2022 - IEEE Conf. Comput. Commun.*, London, United Kingdom, 2022, pp. 1409–1418, doi: 10.1109/INFOCOM48880.2022.9796926.

[10] L. N. Tidjon and F. Khomh, "Reliable malware analysis and detection using topology data analysis–.," 2022, *arXiv:2211.01535*.

[11] F. Ceschin et al., "Fast & Furious–: On the modelling of malware detection as an evolving data stream," *Expert Syst. with Appl.*, vol. 212, 2023, Art. no. 118590.

[12] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multi-tiered hybrid intrusion detection system for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 616–632, Jan. 2022, doi: 10.1109/JIOT.2021.3084796.

[13] L. Liu et al., "ConFlow: Contrast network flow improving class-imbalanced learning in network intrusion detection," in Proc. *SPNCE 2023*, Cham, Switzerland, 2025, pp. 120–136.

[14] L. Yang and A. Shami, "A Transfer learning and optimized cnn based intrusion detection system for internet of vehicles," in *Proc. IEEE ICC*, Seoul, South Korea, 2022, pp. 1–6.

[15] M. Zhong, M. Lin, and Z. He, "Dynamic multi-scale topological representation for enhancing network intrusion detection," *Comput. Secur.*, vol. 135, 2023, Art. no. 103516.

[16] M. Lotfollahi et al., "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, 2020.

[17] I. Sharafaldin, A.H. Lashkari, and A.A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSP*, 2018, vol. 1, pp. 108–116.

[18] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustain. Cities Soc.*, vol. 72, Sep. 2021, Art. no. 102994.

[19] S. Tsimenidis, T. Lagkas, and K. Rantos, "Deep learning in IoT intrusion detection," *J. Netw. Syst. Manage.*, vol. 30, no. 8, pp. 1–40, 2022.

[20] D. Kim et al., "SelfReg: Self-supervised contrastive regularization for domain generalization," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, 9619–9628.

[21] W. Huang et al., "Towards the generalization of contrastive self-supervised learning.," in *Proc. Int. Conf. Learn. Representations*, Kigali, Rwanda, 2023.

[22] L. Le, A. Patterson, and M. White "Supervised autoencoders: Improving generalization performance with unsupervised regularizers," in *Proc. Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 107–117.

[23] I. O. Lopes et al., "Effective network intrusion detection via representation learning: A denoising AutoEncoder approach," *Comput. Commun.*, vol. 194, pp. 55–65, 2022.

[24] X. Liang, Y. Ji, W. -S. Zheng, W. Zuo, and X. Zhu, "SV-Learner: Support-vector contrastive learning for robust learning with noisy labels," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 10, pp. 5409–5422, Oct. 2024.

[25] N. F. Syed, M. Ge, and Z. Baig, "Fog-cloud based intrusion detection system using recurrent neural networks and feature selection for IoT networks," *Comput. Netw.*, vol. 225, 2023, Art. no. 109662 .

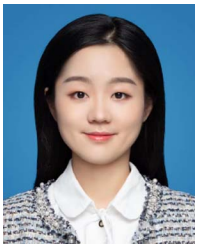
[26] L. Lyu et al., "Fog-embedded deep learning for the Internet of Things," *IEEE Trans. Ind. Inform.*, vol. 15, no. 7, pp. 4206–4215, Jul. 2019.

[27] M. Lopez-Martin et al., "Supervised contrastive learning over prototype-label embeddings for network intrusion detection," *Inf. Fusion*, vol. 79, pp. 200–228, 2022.

[28] Y. Yue, X. Chen, Z. Han, X. Zeng, and Y. Zhu, "Contrastive learning enhanced intrusion detection," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 4, pp. 4232–4247, Dec. 2022, doi: 10.1109/TNSM.2022.3218843.

[29] H. Dong and I. Kotenko, "An autoencoder-based multi-task learning for intrusion detection in IoT networks," in *Proc. IEEE Ural-Siberian Conf. Biomed. Eng., Radioelectronics Inf. Techn.*, Yekaterinburg, Russian Federation, 2023, pp. 1–4.

[30] H. Q. Le, Y. Qiao, L. X. Nguyen, L. Zou, and C. S. Hong, "Federated multimodal learning for IOT applications: A contrastive learning approach," in *Proc. 24th Asia-Pacific Netw. Oper. Manage. Symp.*, sejong, korea, republic, 2023, pp. 201–206.
[31] P. V. Dinh, Q. U. Nguyen, D. T. Hoang, D. N. Nguyen, S. P. Bao, and E. Dutkiewicz, "Constrained twin variational auto-encoder for intrusion detection in IoT systems," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14789–14803, Apr. 2024, doi: 10.1109/JIOT.2023.3344842.
[32] K. He et al., "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020,, pp. 9729–9738, doi: 10.1109/CVPR42600.2020.00975.
[33] Y. Zhao, H. Lu, S. Zhao, H. Wu, and Z. Lu, "Multi-level contrastive learning for hybrid cross-modal retrieval," in *Proc. ICASSP 2024-2024 IEEE Int. Conf. Acoust., Speech Signal Process.*, Seoul, Korea, Republic, 2024,pp. 6390–6394.
[34] H. Deng and Y. Yang, "Context-enriched contrastive loss: Enhancing presentation of inherent sample connections in contrastive learning framework," *IEEE Trans. Multimedia*, vol. 27, pp. 429–441, 2025.
[35] L. Pan and Q. Ren, "Urban traffic flow forecasting based on spatial-temporal graph contrastive learning," in *Proc. ICASSP 2024-2024 IEEE Int. Conf. Acoust., Speech and Signal Process.*, Seoul, Korea, Republic, 2024, pp. 5560–5564.
[36] A. Eijpe et al., "Disentangled and interpretable multimodal attention fusion for cancer survival prediction," in *Proc. MICCAI*, 2025, pp. 117–127.



Xinlei Wang received the PhD degree in electronic science and technology from the Beijing University of Posts and Telecommunications, Beijing, China, in 2025. Her current research interests include machine learning and network security.

Mingshu He (Member, IEEE) received the PhD degree in electronic science and technology from the Beijing University of Posts and Telecommunications, Beijing, China, in 2022. He is currently doing research with the School of Cyberspace Security, Beijing University of Posts and Telecommunications. His research interests include network security, anomaly detection, and machine learning.

Xiaojuan Wang (Member, IEEE) received the PhD degree in electronic science and technology from the University of Beijing University of Posts and Telecommunications, Beijing, China, in 2015. She is currently an associate professor with the School of Electronic Engineering, Beijing University of Posts and Telecommunications. Her research interests include deep learning, complex networks, and human gesture recognition.

Shize Guo was born in 1969. He received the MS and BS degrees from Ordnance Engineering College, China, in 1991 and 1988, respectively, and the PhD degree from the Harbin Institute of Technology, in 1989. He is currently a researcher with the Institute of North Electronic Equipment and also a professor with the Beijing University of Post and Telecommunications. His main research interests include information technology and information security.